

搭配 Vertex AI 和 LangChain4j，以 Java 運用 Gemini

來源：<https://codelabs.developers.google.com/codelabs/gemini-java-developers?hl=zh-tw>

步驟數：14

在這個程式碼研究室中，您將與使用者對話、詢問說明文件相關問題，或透過函式呼叫來擴充模型、使用 Java 中的生成式 AI、在 Vertex AI 中整合 Gemini 大型語言模型，以及運用 LangChain4j 架構。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 準備開發環境](#)
4. [4. 首次呼叫 Gemini 模型](#)
5. [5. 與 Gemini 對話](#)
6. [6. Gemini 的多模態功能](#)
7. [7. 從非結構化文字中擷取結構化資訊](#)
8. [8. 使用提示範本建構提示](#)
9. [9. 使用少量樣本提示法進行文字分類](#)
10. [10. 檢索增強生成](#)
11. [11. 函式呼叫](#)
12. [12. LangChain4j 會處理函式呼叫](#)
13. [13. 使用 Ollama 和 TestContainers 執行 Gemma](#)
14. [14. 恭喜](#)

1. 簡介

本程式碼研究室著重於 Google Cloud [Vertex AI](#) 上託管的 [Gemini](#) 大型語言模型 (LLM)。Vertex AI 平台涵蓋 Google Cloud 上的所有機器學習產品、服務和模型。

您將使用 Java，透過 [LangChain4j](#) 架構與 [Gemini API](#) 互動。您將透過具體範例，瞭解如何運用 LLM 回答問題、生成點子、擷取實體和結構化內容、檢索增強生成，以及呼叫函式。

什麼是生成式 AI？

生成式 AI 是指使用人工智慧生成新內容，例如文字、圖片、音樂、音訊和影片。

生成式 AI 由大型語言模型 (LLM) 驅動，可同時處理多項工作，並立即執行各種任務，包括提供摘要、問與答和分類等。基礎模型只需要少量訓練，就能以極少的範例資料針對指定用途調整。

生成式 AI 的運作方式為何？

生成式 AI 的運作方式是使用機器學習 (ML) 模型，根據一系列人類創作內容的資料來掌握相關模式與關係，再運用學到的模式生成新內容。

訓練生成式 AI 模型最常見的方法是監督式學習。向模型提供一系列人類創作內容和相應的標籤，讓模型學習生成與人類創作內容相似的內容。

常見的生成式 AI 應用程式有哪些？

生成式 AI 有助於：

- 增強即時通訊與搜尋體驗，藉此改善與顧客的互動方式。
- 運用對話式介面和摘要功能，探索大量非結構化資料。
- 協助處理重複性工作，例如回覆提案請求、以不同語言將行銷內容本地化，以及確認客戶合約是否符合相關法規等。

Google Cloud 提供哪些生成式 AI 產品？

有了 [Vertex AI](#)，您就能與基礎模型互動、自訂模型，並將模型嵌入應用程式中，而且不需要具備機器學習專業知識，也能輕鬆上手。您可以在 [Model Garden](#) 中存取基礎模型、透過 [Vertex AI Studio](#) 的簡易使用者介面調整模型，或是運用數據資料學筆記本中的模型。

有了 [Vertex AI Search and Conversation](#)，開發人員就能在最短時間內，建構生成式 AI 技術輔助[搜尋引擎](#)和聊天機器人。

[Google Cloud 專用 Gemini](#) 採用 Gemini 技術，是 AI 輔助協作工具，在 Google Cloud 和 IDE 中皆可使用，能協助您以更快的速度完成更多工作。[Gemini Code Assist](#) 可提供程式碼補全、程式碼生成、程式碼說明功能，並支援對話，方便您提出技術問題。

Gemini 是什麼？

Gemini 是由 Google DeepMind 開發的一系列生成式 AI 模型，專為多模態用途而設計。多模態代表可以處理及生成不同類型的內容，例如文字、程式碼、圖片和音訊。



Gemini 有多種變體和大小：

- **Gemini 2.0 Flash**：我們最新推出的新一代功能，效能更上一層樓。
- **Gemini 2.0 Flash-Lite**：這是 Gemini 2.0 Flash 模型，延遲時間最短，成本效益最高。
- **Gemini 2.5 Pro**：這是我們目前最先進的推理模型。
- **Gemini 2.5 Flash**：思考模型，功能全面。兼顧價格與效能。

主要功能與特色：

- **多模態**：Gemini 不僅能解讀文字，還能理解及處理多種資訊格式，這項能力遠勝於傳統的純文字語言模型。
- **效能**：Gemini 2.5 Pro 在許多基準評測中都優於目前最先進的模型，也是第一個在難度極高的 MMLU (大規模多工作語言理解) 基準評測中，表現超越人類專家的模型。
- **彈性**：Gemini 有多種尺寸，可因應各種用途調整，從大規模研究到在行動裝置上部署都適用。

如何透過 Java 與 Vertex AI 的 Gemini 互動？

方法有以下兩種：

1. 官方 [Vertex AI Java API for Gemini](#) 程式庫。
2. [LangChain4j](#) 架構。

在本程式碼研究室中，您將使用 [LangChain4j](#) 架構。

什麼是 LangChain4j 架構？

[LangChain4j](#) 框架是開放原始碼程式庫，可透過自動調度管理各種元件 (例如 LLM 本身，以及向量資料庫 (用於語意搜尋)、文件載入器和分割器 (用於分析文件並從中學習)、輸出剖析器等其他工具)，在 Java 應用程式中整合 LLM。

這個專案的靈感來自 [LangChain](#) Python 專案，但目標是為 Java 開發人員提供服務。

從 [Github 專案頁面](#)：

這個專案的目標是簡化將 AI/LLM 功能整合至 Java 應用程式的程序。

這要歸功於：

- **簡單且連貫的抽象層**，可確保程式碼不會依附於具體實作內容，例如 LLM 供應商、嵌入式儲存空間供應商等。這樣一來，您就能輕鬆更換元件。
- **上述抽象概念的眾多實作項目**，提供多種 LLM 和嵌入式儲存空間供您選擇。
- **熱門功能**，例如：
 - **匯入自有資料** (文件、程式碼集等)，讓 LLM 根據您的資料採取行動及回覆。
 - **自主代理**：將 (即時定義的) 工作委派給 LLM，並盡力完成工作。
 - **提示範本**：協助您盡可能提高 LLM 回覆的品質。
 - **記憶功能**可為 LLM 提供當前和先前的對話脈絡。
 - **結構化輸出內容**：以 Java POJO 形式接收 LLM 回覆，並取得所需結構。
 - **「AI 服務」**：用於以宣告方式定義簡單 API 背後複雜的 AI 行為。
 - **鏈結**：減少常見用途中大量樣板程式碼的需求。
 - **自動審查**，確保 LLM 的所有輸入和輸出內容都不會造成危害。



課程內容

- 如何設定 Java 專案，以便使用 Gemini 和 LangChain4j
- 如何以程式輔助方式將第一個提示詞傳送至 Gemini

- 如何啟用 Gemini 逐句顯示回覆
- 如何建立使用者與 Gemini 之間的對話
- 如何傳送文字和圖片，在多模態情境中使用 Gemini
- 如何從非結構化內容中擷取實用的結構化資訊
- 如何操作提示範本
- 如何執行文字分類，例如情緒分析
- 如何與自己的文件對話 (檢索增強生成)
- 如何透過函式呼叫擴充聊天機器人功能
- 如何透過 Ollama 和 TestContainers 在本機使用 Gemma

軟硬體需求

- 熟悉 Java 程式設計語言
 - 具備 Google Cloud 專案
 - 瀏覽器，例如 Chrome 或 Firefox
-

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

啟用 Cloud Shell

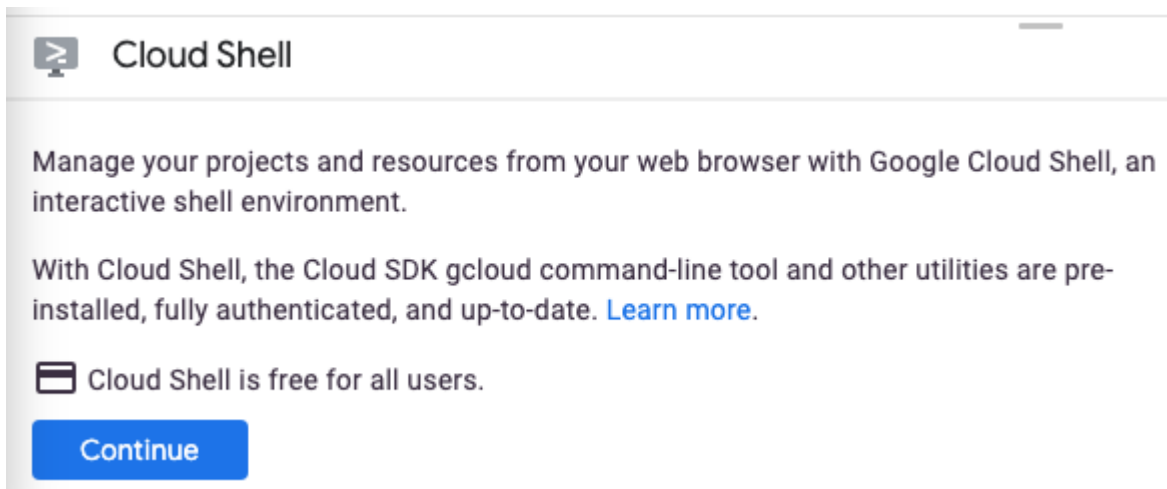
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



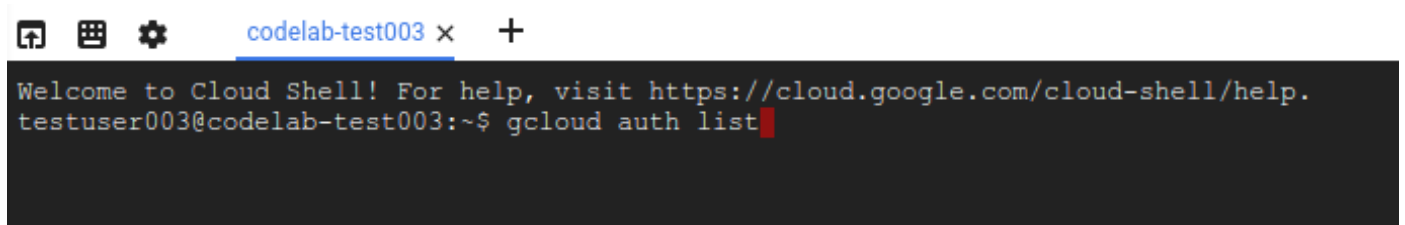
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 5 分鐘

3. 準備開發環境

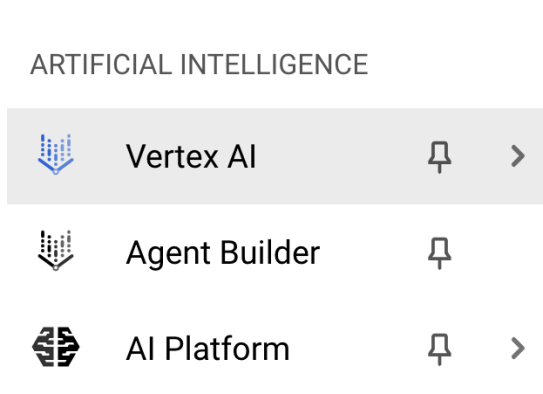
在本程式碼研究室中，您將使用 Cloud Shell 終端機和 Cloud Shell 編輯器開發 Java 程式。

啟用 Vertex AI API

在 [Google Cloud 控制台](#) 中，確認專案名稱顯示在頂端。如果不是，請點按「選取專案」開啟**專案選取器**，然後選取所需專案。

您可以從 Google Cloud 控制台的 Vertex AI 專區或 Cloud Shell 終端機啟用 Vertex AI API。

如要透過 Google Cloud 控制台啟用，請先前往 Google Cloud 控制台選單的 Vertex AI 專區：



在 Vertex AI 資訊主頁中，點按「啟用所有建議的 API」。

資訊：啟用程序可能需要一些時間才能完成。啟用 API 時，Google Cloud 控制台右上方的鈴鐺圖示會顯示藍色圓圈。

這會啟用多個 API，但本程式碼研究室最重要的 API 是 `aiplatform.googleapis.com`。

或者，您也可以 Cloud Shell 終端機中執行下列指令，啟用這項 API：

```
gcloud services enable aiplatform.googleapis.com
```

複製 Github 存放區

在 Cloud Shell 終端機中，複製本程式碼研究室的存放區：

```
git clone https://github.com/glaforge/gemini-workshop-for-java-developers.git
```

如要確認專案是否已準備好執行，可以試著執行「Hello World」程式。

確認您位於頂層資料夾：

```
cd gemini-workshop-for-java-developers/
```

建立 Gradle 包裝函式：

```
gradle wrapper
```

使用 `gradlew` 執行：

```
./gradlew run
```

您應該會看到以下的輸出內容：

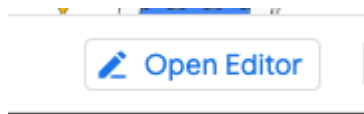
```
..  
> Task :app:run  
Hello World!
```

注意：我們使用 Java 21 編譯範例，但這些範例應可搭配舊版 Java 運作。如果 Java 21 不是 Cloud Shell 環境中安裝的預設版本，且您想使用 Java 21，可以執行下列指令將其設為預設版本：

```
sudo update-java-alternatives --set java-1.21.0-openjdk-amd64
```

開啟及設定 Cloud Editor

從 Cloud Shell 開啟 **Cloud Code 編輯器** 中的程式碼：



在 Cloud Code 編輯器中，依序選取 `File -> Open Folder`，然後指向 Codelab 來源資料夾 (例如 `/home/username/gemini-workshop-for-java-developers/`)。

設定環境變數

在 Cloud Code 編輯器中，依序選取 `Terminal -> New Terminal`，開啟新的終端機。設定執行程式碼範例所需的兩個環境變數：

- **PROJECT_ID**：您的 Google Cloud 專案 ID
- **LOCATION**：Gemini 模型部署的區域

匯出變數，如下所示：

```
export PROJECT_ID=$(gcloud config get-value project)  
export LOCATION=us-central1
```

警告：如果 Cloud Shell 工作階段逾時，重新連線時必須匯出這兩個環境變數，因為 Shell 不會記住這些變數。

步驟 4 · 約 3 分鐘

4. 首次呼叫 Gemini 模型

專案設定完成後，就可以呼叫 Gemini API 了。

查看 `app/src/main/java/gemini/workshop` 目錄中的 `QA.java`：

```
package gemini.workshop;

import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.model.chat.ChatLanguageModel;

public class QA {
    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .build();

        System.out.println(model.generate("Why is the sky blue?"));
    }
}
```

在第一個範例中，您需要匯入 `VertexAiGeminiChatModel` 類別，該類別會實作 `ChatModel` 介面。

在 `main` 方法中，您可以使用 `VertexAiGeminiChatModel` 的建構工具設定即時通訊語言模型，並指定：

- 專案
- 位置
- 模型名稱 (`gemini-2.0-flash`)。

語言模型準備就緒後，您就可以呼叫 `generate()` 方法，並傳遞提示、問題或指令，傳送至 LLM。在這裡，你問了一個簡單的問題，瞭解天空為何是藍色。

你可以隨意變更提示，嘗試不同的問題或工作。

在原始碼根資料夾中執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.QA
```

畫面會顯示類似如下的輸出：

The sky appears blue because of a phenomenon called Rayleigh scattering. When sunlight enters the atmosphere, it is made up of a mixture of different wavelengths of light, each with a different color. The different wavelengths of light interact with the molecules and particles in the atmosphere in different ways.

The shorter wavelengths of light, such as those corresponding to blue and violet light, are more likely to be scattered in all directions by these particles than the longer wavelengths of light, such as those corresponding to red and orange light. This is because the shorter wavelengths of light have a smaller wavelength and are able to bend around the particles more easily.

As a result of Rayleigh scattering, the blue light from the sun is scattered in all directions, and it is this scattered blue light that we see when we look up at the sky. The blue light from the sun is not actually scattered in a single direction, so the color of the sky can vary depending on the position of the sun in the sky and the amount of dust and water droplets in the atmosphere.

恭喜，您已向 Gemini 發出第一通呼叫！

逐句顯示回覆

你是否發現幾秒後，系統就一次給出回覆？此外，您也可以透過串流回應變體，逐步取得回應。串流回應：模型會逐一傳回回應，直到全部傳回為止。

在本程式碼研究室中，我們將使用非串流回應，但會查看串流回應，瞭解如何執行這項操作。

在 `app/src/main/java/gemini/workshop` 目錄的 `StreamQA.java` 中，您可以看到串流回應的運作情形：

```
package gemini.workshop;

import dev.langchain4j.model.chat.StreamingChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiStreamingChatModel;

import static dev.langchain4j.model.LambdaStreamingResponseHandler.onNext;

public class StreamQA {
    public static void main(String[] args) {
        StreamingChatLanguageModel model = VertexAiGeminiStreamingChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .maxOutputTokens(4000)
            .build();

        model.generate("Why is the sky blue?", onNext(System.out::println));
    }
}
```

這次我們匯入實作 `StreamingChatLanguageModel` 介面的串流類別變數 `VertexAiGeminiStreamingChatModel`。您也需要靜態匯入 `LambdaStreamingResponseHandler.onNext`，這是提供 `StreamingResponseHandler` 的便利方法，可使用 Java lambda 運算式建立串流處理常式。

這次 `generate()` 方法的簽章略有不同。傳回型別為 `void`，而非字串。除了提示之外，您還必須傳遞串流回應處理常式。在此，由於我們已完成上述靜態匯入作業，因此可以定義要傳遞至 `onNext()` 方法的 lambda 運算式。每當有新的回應片段可用時，系統就會呼叫 lambda 運算式，而後者只會在發生錯誤時呼叫。

注意：在上述範例中，我們使用 Java 的方法參照標記法，這實際上是 lambda 運算式。但您也可以寫入 `(text) -> { System.out.println(text); }`，而非 `System.out::println`。除了接受這些 lambda 運算式的 `onNext()` 靜態方法外，還有 `onNextAndError()` 變體，可接受兩個運算式：一個用於接受文字，另一個用於處理可能發生的錯誤。

執行作業：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.StreamQA
```

您會收到與前一個類別類似的答案，但這次您會發現答案會逐步顯示在殼層中，而不是等待完整答案顯示。

額外設定

就設定而言，我們只定義了專案、位置和模型名稱，但您還可以為模型指定其他參數：

- `temperature(Float temp)`：定義回覆的創意程度 (0 代表創意度較低，通常更符合事實；2 代表創意度較高)
- `topP(Float topP)`：選取總機率加總為該浮點數 (介於 0 和 1 之間) 的可能字詞
- `topK(Integer topK)`：從文字完成功能最多可用的可能字詞中隨機選取字詞 (1 到 40)
- `maxOutputTokens(Integer max)`：指定模型回覆的長度上限 (一般來說，4 個權杖約等於 3 個字)
- `maxRetries(Integer retries)` - 如果您超出每次要求的配額，或平台發生技術問題，模型可以重試呼叫 3 次

資訊：如要進一步瞭解 `temperature`、`Top-P`、`Top-K` 和 `max token` 值的確切意義，請參閱[測試文字提示](#)的說明文件頁面。

目前你只向 Gemini 提問一次，但你也可以進行多輪對話。這就是下一節要探討的主題。

步驟 5 · 約 4 分鐘

5. 與 Gemini 對話

在上一個步驟中，您只問了一個問題。現在，使用者可以與 LLM 進行實際對話。每個問題和答案都可以以前一個為基礎，形成真正的討論。

查看 `app/src/main/java/gemini/workshop` 資料夾中的 `Conversation.java`：

```
package gemini.workshop;

import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.memory.chat.MessageWindowChatMemory;
import dev.langchain4j.service.AiServices;

import java.util.List;

public class Conversation {
    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .build();

        MessageWindowChatMemory chatMemory = MessageWindowChatMemory.builder()
            .maxMessages(20)
            .build();

        interface ConversationService {
            String chat(String message);
        }

        ConversationService conversation =
            AiServices.builder(ConversationService.class)
                .chatLanguageModel(model)
                .chatMemory(chatMemory)
                .build();

        List.of(
            "Hello!",
            "What is the country where the Eiffel tower is situated?",
            "How many inhabitants are there in that country?"
        ).forEach( message -> {
            System.out.println("\nUser: " + message);
            System.out.println("Gemini: " + conversation.chat(message));
        });
    }
}
```

這個類別中有幾個有趣的新匯入項目：

- `MessageWindowChatMemory`：這個類別有助於處理對話的多輪互動，並將先前的問題和答案保留在本地記憶體中
- `AiServices`：較高層級的抽象類別，可將聊天模型和聊天記憶體繫結在一起

在主要方法中，您將設定模型、對話記憶體和 AI 服務。模型會照常設定專案、位置和模型名稱資訊。

在對話記憶體方面，我們使用 `MessageWindowChatMemory` 的建構工具建立記憶體，保留最近 20 則訊息。這是對話的滑動視窗，其脈絡會保留在 Java 類別用戶端中。

注意：LLM 是無狀態模型，他們不會追蹤對話內容，而是每次附加新問題時，都必須將整個對話傳回。幸好 `LangChain4j` 會為我們處理這方面的事宜。這裡我們任意選擇保留 20 則訊息，確保您不會將所有內容都保留在記憶體中。LLM 的處理能力有限 (即「脈絡窗口」)。根據脈絡視窗大小和對話的通常長度調整訊息數量。此外，請注意，傳送給 LLM 的權杖越多，費用就越高！

接著，您會建立 `AI service`，將對話模型與對話記憶體繫結。

請注意，AI 服務如何使用我們定義的自訂 `ConversationService` 介面 (`LangChain4j` 會實作該介面)，並接受 `String` 查詢和傳回 `String` 回應。

注意：您可以決定這個介面的外觀。通常您只需定義一個介面，其中包含一個方法，該方法會接收字串 (使用者的提示)，並傳回回應字串。不過，`LangChain4j` 支援更精細的簽章。如要進一步瞭解 `AiServices` 類別，請參閱專案的[說明文件](#)。

現在，你可以與 Gemini 對話。首先，系統會傳送簡單的問候訊息，然後提出第一個問題，詢問艾菲爾鐵塔位於哪個國家/地區。請注意，最後一句與第一個問題的答案有關，因為您想知道艾菲爾鐵塔所在國家/地區的居民人數，但沒有明確提及先前答案中提供的國家/地區。這表示系統會在每次提示時傳送先前的問題和答案。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.Conversation
```

畫面上應會顯示類似以下的三個答案：

User: Hello!

Gemini: Hi there! How can I assist you today?

User: What is the country where the Eiffel tower is situated?

Gemini: France

User: How many inhabitants are there in that country?

Gemini: As of 2023, the population of France is estimated to be around 67.8 million.

注意：如果收到 RESOURCE_EXHAUSTED 錯誤，表示您已超出 VertexAI API 的配額。您可以點選錯誤訊息中的連結來提高配額，或嘗試使用其他模型。

試用 Gemini 的 Chat 記憶功能，讓對話更順暢。在 `List.of` 方法中新增問題。例如，在詢問居民人數的問題後插入以下句子 `What are the colors of the flag of this country?`。重新執行程式碼，你會發現 Gemini 會從聊天室記憶體的脈絡推斷你的問題與法國有關，並回覆法國國旗的顏色。

你可以向 Gemini 提出單輪問題，或進行多輪對話，但目前只能輸入文字。圖片呢？我們會在下一個步驟中探索圖片。

步驟 6 · 約 4 分鐘

6. Gemini 的多模態功能

Gemini 是多模態模型，不僅能接受文字輸入，還能接受圖片或影片輸入。在本節中，您會看到混合使用文字和圖片的應用案例。

你認為 Gemini 能認出這隻貓嗎？



取自 [Wikipedia](#) 的雪地貓咪圖片

查看 `app/src/main/java/gemini/workshop` 目錄中的 `Multimodal.java`：

```

package gemini.workshop;

import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.data.message.UserMessage;
import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.model.output.Response;
import dev.langchain4j.data.message.ImageContent;
import dev.langchain4j.data.message.TextContent;

public class Multimodal {

    static final String CAT_IMAGE_URL =
        "https://upload.wikimedia.org/wikipedia/" +
        "commons/b/b6/Felis_catus-cat_on_snow.jpg";

    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .build();

        UserMessage userMessage = UserMessage.from(
            ImageContent.from(CAT_IMAGE_URL),
            TextContent.from("Describe the picture")
        );

        Response<AiMessage> response = model.generate(userMessage);

        System.out.println(response.content().text());
    }
}

```

在匯入作業中，我們會區分不同類型的訊息和內容。`UserMessage` 可以同時包含 `TextContent` 和 `ImageContent` 物件。這就是多模態的應用：混搭文字和圖像。我們不會只傳送簡單的字串提示，而是傳送代表使用者訊息的結構化物件，其中包含圖片內容和文字內容。模型會傳回包含 `AiMessage` 的 `Response`。

然後，您透過 `content()` 從回應中擷取 `AiMessage`，再透過 `text()` 擷取訊息文字。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.Multimodal
```

圖片名稱確實暗示了圖片內容，但 Gemini 輸出內容類似於以下內容：

```
A cat with brown fur is walking in the snow. The cat has a white patch of fur on its chest and white paws. The cat is looking at the camera.
```

混合使用圖片和文字提示，可帶來許多有趣的用途。您可以建立的應用程式包括：

- 辨識圖片中的文字。
- 檢查圖片是否可安全顯示。
- 製作圖片說明。
- 使用純文字說明搜尋圖片資料庫。

除了從圖片擷取資訊，您也可以從非結構化文字擷取資訊。這就是下一節要學習的內容。

7. 從非結構化文字中擷取結構化資訊

在許多情況下，重要資訊會以非結構化方式出現在報告文件、電子郵件或其他長篇文字中。理想情況下，您希望能夠以結構化物件的形式，從非結構化文字中擷取重要詳細資料。接著就來看看實際做法。

資訊：一般估計，結構化資料 (例如資料庫表格、試算表等) 約占資料的 20%，而 80% 的資訊都是非結構化資料。

假設您想從某人的自傳、履歷或描述中擷取姓名和年齡，您可以透過巧妙調整的提示詞，指示 LLM 從非結構化文字中擷取 JSON (這通常稱為「提示詞工程」)。

但在下列範例中，我們不會編寫提示來描述 JSON 輸出內容，而是使用 Gemini 的強大[功能](#)，也就是「結構化輸出」(有時也稱為受限生成)，強制模型只輸出符合指定 [JSON 結構定義](#) 的有效 JSON 內容。

請參閱 `app/src/main/java/gemini/workshop` 中的 [ExtractData.java](#)：

```

package gemini.workshop;

import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.service.AiServices;
import dev.langchain4j.service.SystemMessage;

import static dev.langchain4j.model.vertexai.SchemaHelper.fromClass;

public class ExtractData {

    record Person(String name, int age) { }

    interface PersonExtractor {
        @SystemMessage("""
            Your role is to extract the name and age
            of the person described in the biography.
            """)
        Person extractPerson(String biography);
    }

    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .responseMimeType("application/json")
            .responseSchema(fromClass(Person.class))
            .build();

        PersonExtractor extractor = AiServices.create(PersonExtractor.class, model);

        String bio = """
            Anna is a 23 year old artist based in Brooklyn, New York. She was born and
            raised in the suburbs of Chicago, where she developed a love for art at a
            young age. She attended the School of the Art Institute of Chicago, where
            she studied painting and drawing. After graduating, she moved to New York
            City to pursue her art career. Anna's work is inspired by her personal
            experiences and observations of the world around her. She often uses bright
            colors and bold lines to create vibrant and energetic paintings. Her work
            has been exhibited in galleries and museums in New York City and Chicago.
            """;

        Person person = extractor.extractPerson(bio);

        System.out.println(person.name()); // Anna
        System.out.println(person.age()); // 23
    }
}

```

我們來看看這個檔案中的各個步驟：

- **Person** 記錄定義為代表描述某人 (姓名和年齡) 的詳細資料。

- `PersonExtractor` 介面定義的方法會傳回 `Person` 執行個體，並提供非結構化文字字串。
- `extractPerson()` 會加上 `@SystemMessage` 註解，將指令提示與其建立關聯。模型會使用這個提示引導資訊擷取作業，並以 JSON 文件形式傳回詳細資料，這些資料會經過剖析，並解封送至 `Person` 例項。

現在來看看 `main()` 方法的內容：

- 設定並例項化對話模型。我們使用了模型建構工具類別的 2 個新方法：`responseMimeType()` 和 `responseSchema()`。第一個提示會要求 Gemini 在輸出內容中生成有效的 JSON。第二種方法會定義應傳回的 JSON 物件結構定義。此外，後者會委派給便利方法，可將 Java 類別或記錄轉換為適當的 JSON 結構定義。
- `PersonExtractor` 物件是透過 LangChain4j 的 `AiServices` 類別建立。
- 接著，您只要呼叫 `Person person = extractor.extractPerson(...)`，即可從非結構化文字中擷取人員詳細資料，並取得含有姓名和年齡的 `Person` 執行個體。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.ExtractData
```

您應該會看到以下的輸出內容：

```
Anna  
23
```

是，這是 Anna，她 23 歲！

使用這種 `AiServices` 方法時，您會使用強型別物件。您不會直接與 LLM 互動，而是使用具體類別，例如代表擷取個人資訊的 `Person` 記錄，以及具有 `extractPerson()` 方法的 `PersonExtractor` 物件，該方法會傳回 `Person` 例項。LLM 的概念已抽象化，而身為 Java 開發人員，您在使用這個 `PersonExtractor` 介面時，只需操作一般類別和物件。

8. 使用提示範本建構提示

使用一組常見的指令或問題與 LLM 互動時，提示中有一部分永遠不會變更，其他部分則包含資料。舉例來說，如要建立食譜，可以輸入「你是一位才華洋溢的廚師，請使用下列食材製作食譜：...」，然後在文字結尾加上食材。提示範本就是為此而生，類似於程式設計語言中的插補字串。提示範本包含預留位置，您可以將這些位置替換為特定 LLM 呼叫的正確資料。

具體來說，我們來研究 `app/src/main/java/gemini/workshop` 目錄中的 `TemplatePrompt.java`：


```

package gemini.workshop;

import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.model.input.Prompt;
import dev.langchain4j.model.input.PromptTemplate;
import dev.langchain4j.model.output.Response;

import java.util.HashMap;
import java.util.Map;

public class TemplatePrompt {
    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .maxOutputTokens(500)
            .temperature(1.0f)
            .topK(40)
            .topP(0.95f)
            .maxRetries(3)
            .build();

        PromptTemplate promptTemplate = PromptTemplate.from("""
            You're a friendly chef with a lot of cooking experience.
            Create a recipe for a {{dish}} with the following ingredients: \
            {{ingredients}}, and give it a name.
            """)
        );

        Map<String, Object> variables = new HashMap<>();
        variables.put("dish", "dessert");
        variables.put("ingredients", "strawberries, chocolate, and whipped cream");

        Prompt prompt = promptTemplate.apply(variables);

        Response<AiMessage> response = model.generate(prompt.toUserMessage());

        System.out.println(response.content().text());
    }
}

```

如常設定 `VertexAiGeminiChatModel` 模型，並使用高溫、高 `topP` 和 `topK` 值，提高創意程度。接著，您可以使用 `from()` 靜態方法建立 `PromptTemplate`，方法是傳遞提示字串，並使用雙大括號預留位置變數：`{{dish}}` 和 `{{ingredients}}`。

您可呼叫 `apply()` 建立最終提示，這會採用鍵/值組合對應，代表預留位置名稱和要取代的字串值。

最後，您要從該提示詞建立使用者訊息，並使用 `prompt.toUserMessage()` 指令，呼叫 Gemini 模型的 `generate()` 方法。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.TemplatePrompt
```

您應該會看到類似以下的生成輸出內容：

****Strawberry Shortcake****

Ingredients:

- * 1 pint strawberries, hulled and sliced
- * 1/2 cup sugar
- * 1/4 cup cornstarch
- * 1/4 cup water
- * 1 tablespoon lemon juice
- * 1/2 cup heavy cream, whipped
- * 1/4 cup confectioners' sugar
- * 1/4 teaspoon vanilla extract
- * 6 graham cracker squares, crushed

Instructions:

1. In a medium saucepan, combine the strawberries, sugar, cornstarch, water, and lemon juice. Bring to a boil over medium heat, stirring constantly. Reduce heat and simmer for 5 minutes, or until the sauce has thickened.
2. Remove from heat and let cool slightly.
3. In a large bowl, combine the whipped cream, confectioners' sugar, and vanilla extract. Beat until soft peaks form.
4. To assemble the shortcakes, place a graham cracker square on each of 6 dessert plates. Top with a scoop of whipped cream, then a spoonful of strawberry sauce. Repeat layers, ending with a graham cracker square.
5. Serve immediately.

****Tips:****

- * For a more elegant presentation, you can use fresh strawberries instead of sliced strawberries.
- * If you don't have time to make your own whipped cream, you can use store-bought whipped cream.

您可以隨意變更對應地圖中的 `dish` 和 `ingredients` 值，並調整溫度、`topK` 和 `topP`，然後重新執行程式碼。這樣您就能觀察變更這些參數對 LLM 的影響。

提示範本是個好方法，可讓您重複使用及參數化 LLM 呼叫的指令。您可以傳遞資料，並根據使用者提供的值自訂提示。

9. 使用少量樣本提示法進行文字分類

大型語言模型很擅長將文字分類到不同類別。您可以提供一些文字範例及其相關聯的類別，協助 LLM 完成這項工作。這種做法通常稱為「少量樣本提示」。

注意：提供這類範例給 LLM 的技術有許多種：

- 其中一種做法是建立提示範本 (如上一節所述)，並列出輸入和輸出內容。
- 另一種做法是建立 `UserMessage` / `AiMessage` 配對的交替，並將文字傳遞至分類。
- 但在本節中，我們將探討第三種可能性，也就是使用我們已用過的 `AiServices` 較高層級抽象化，以及包含輸入 / 輸出範例的對話記憶體。

這篇文章會實際展示各種技術，以實作情緒分析 (本節中的範例)，另一篇文章則會說明如何使用嵌入模型，透過 `LangChain4j` 內建的文字分類功能，但我們不會在本研討會中介紹這項功能。

讓我們開啟 `app/src/main/java/gemini/workshop` 目錄中的 `TextClassification.java`，執行特定類型的文字分類：情緒分析。

```

package gemini.workshop;

import com.google.cloud.vertexai.api.Schema;
import com.google.cloud.vertexai.api.Type;
import dev.langchain4j.data.message.UserMessage;
import dev.langchain4j.memory.chat.MessageWindowChatMemory;
import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.vertexai.VertexAiGeminiChatModel;
import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.service.AiServices;
import dev.langchain4j.service.SystemMessage;

import java.util.List;

public class TextClassification {

    enum Sentiment { POSITIVE, NEUTRAL, NEGATIVE }

    public static void main(String[] args) {
        ChatLanguageModel model = VertexAiGeminiChatModel.builder()
            .project(System.getenv("PROJECT_ID"))
            .location(System.getenv("LOCATION"))
            .modelName("gemini-2.0-flash")
            .maxOutputTokens(10)
            .maxRetries(3)
            .responseSchema(Schema.newBuilder()
                .setType(Type.STRING)
                .addAllEnum(List.of("POSITIVE", "NEUTRAL", "NEGATIVE"))
                .build())
            .build();

        interface SentimentAnalysis {
            @SystemMessage("""
                Analyze the sentiment of the text below.
                Respond only with one word to describe the sentiment.
                """)
            Sentiment analyze(String text);
        }

        MessageWindowChatMemory memory = MessageWindowChatMemory.withMaxMessages(10);
        memory.add(UserMessage.from("This is fantastic news!"));
        memory.add(AiMessage.from(Sentiment.POSITIVE.name()));

        memory.add(UserMessage.from("Pi is roughly equal to 3.14"));
        memory.add(AiMessage.from(Sentiment.NEUTRAL.name()));

        memory.add(UserMessage.from("I really disliked the pizza. Who would use pineapples as a pizza topping?"));
        memory.add(AiMessage.from(Sentiment.NEGATIVE.name()));

        SentimentAnalysis sentimentAnalysis =
    
```

```
        AiServices.builder(SentimentAnalysis.class)
            .chatLanguageModel(model)
            .chatMemory(memory)
            .build();

        System.out.println(sentimentAnalysis.analyze("I love strawberries!"));
    }
}
```

`Sentiment` 列舉會列出情緒的不同值：負面、中性或正面。

在 `main()` 方法中，您會照常建立 Gemini Chat 模型，但最大輸出權杖數量較少，因為您只需要簡短的回覆：文字為 `POSITIVE`、`NEGATIVE` 或 `NEUTRAL`。如要限制模型只傳回這些值，您可以運用在資料擷取部分發現的結構化輸出支援功能。因此使用 `responseSchema()` 方法。這次您不會使用 `SchemaHelper` 中的便利方法來推斷結構定義，而是使用 `Schema` 建構工具，瞭解結構定義的樣貌。

設定模型後，您會建立 `□` 介面，LangChain4j 的 `□` 會使用 LLM 為您實作該介面。`SentimentAnalysis AiServices` 這個介面包含一個方法：`analyze()`。這項函式會將要分析的文字做為輸入內容，並傳回 `Sentiment` 列舉值。因此，您只會操作代表所辨識情緒類別的強型別物件。

接著，為了提供「少樣本範例」，引導模型執行分類工作，請建立對話記憶體，傳遞成對的使用者訊息和 AI 回覆，代表文字和相關情緒。

讓我們使用 `AiServices.builder()` 方法將所有內容繫結在一起，方法是傳遞 `SentimentAnalysis` 介面、要使用的模型，以及包含少量範例的對話記憶體。最後，呼叫 `analyze()` 方法並傳入要分析的文字。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.TextClassification
```

畫面上應會顯示單一字詞：

POSITIVE

看來喜歡草莓是正向情緒！

10. 檢索增強生成

LLM 經過大量文字訓練，不過，這些模型只能提供訓練期間看過的資訊。如果模型訓練截止日後發布了新資訊，模型就無法取得這些詳細資料。因此，模型無法回答未曾看過的問題。

因此，本節將介紹**檢索增強生成 (RAG)** 等方法，協助提供 LLM 可能需要瞭解的額外資訊，以滿足使用者要求，並回覆訓練時無法存取的最新資訊或私人資訊。

讓我們回到對話。這次你可以詢問文件相關問題。您將建構一個聊天機器人，能夠從資料庫中擷取相關資訊 (資料庫包含您以較小片段 (「區塊」) 分割的文件)，模型會使用這些資訊做為回覆依據，而不是只依賴訓練時學到的知識。

為什麼文件會分割成「區塊」？

因為 RAG 技術會將使用者提示與文件中的每個文字區塊進行比較。如要進行比較，系統會將文字和提示轉換為向量嵌入項目，也就是浮點值的高多維度向量。接著，系統會使用不同的距離或相似度指標比較向量。向量資料庫最常使用的指標是**餘弦相似度**。向量越相似，語意通常也越相關。

區塊大小應為多少？

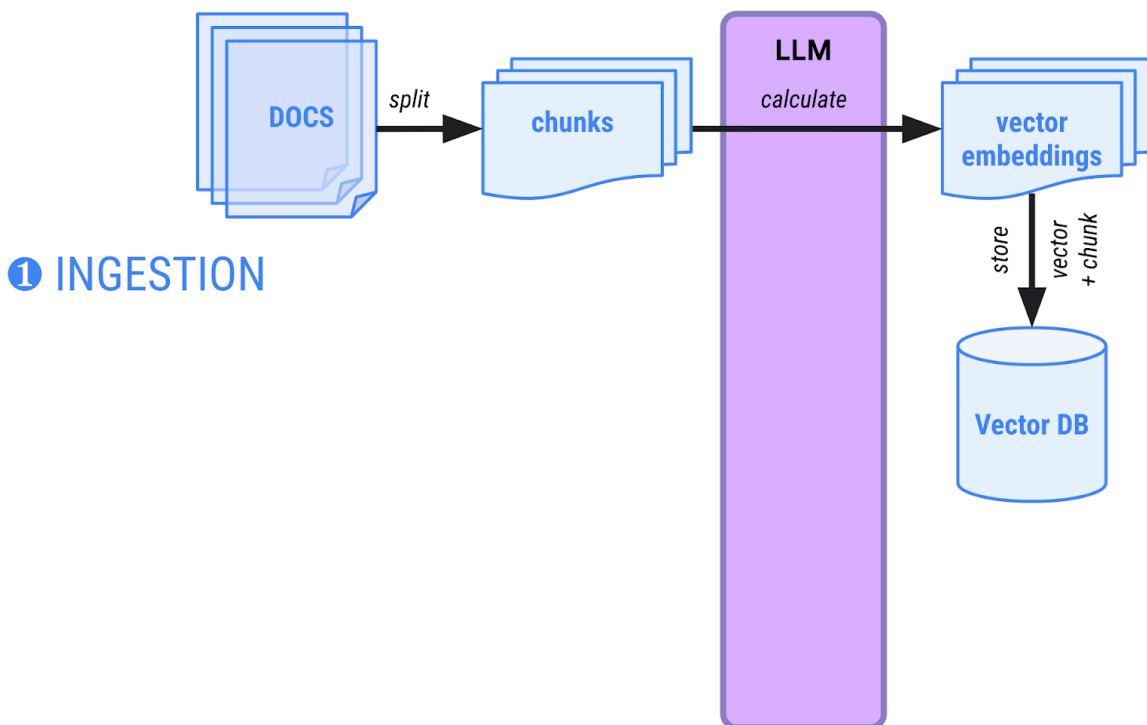
沒有最佳尺寸。建議您嘗試不同的區塊大小，看看是否能為要分析的文件類型產生更優質的結果。區塊大小通常介於 100 到 2000 個字元。

什麼是向量資料庫？我不能使用一般資料庫嗎？

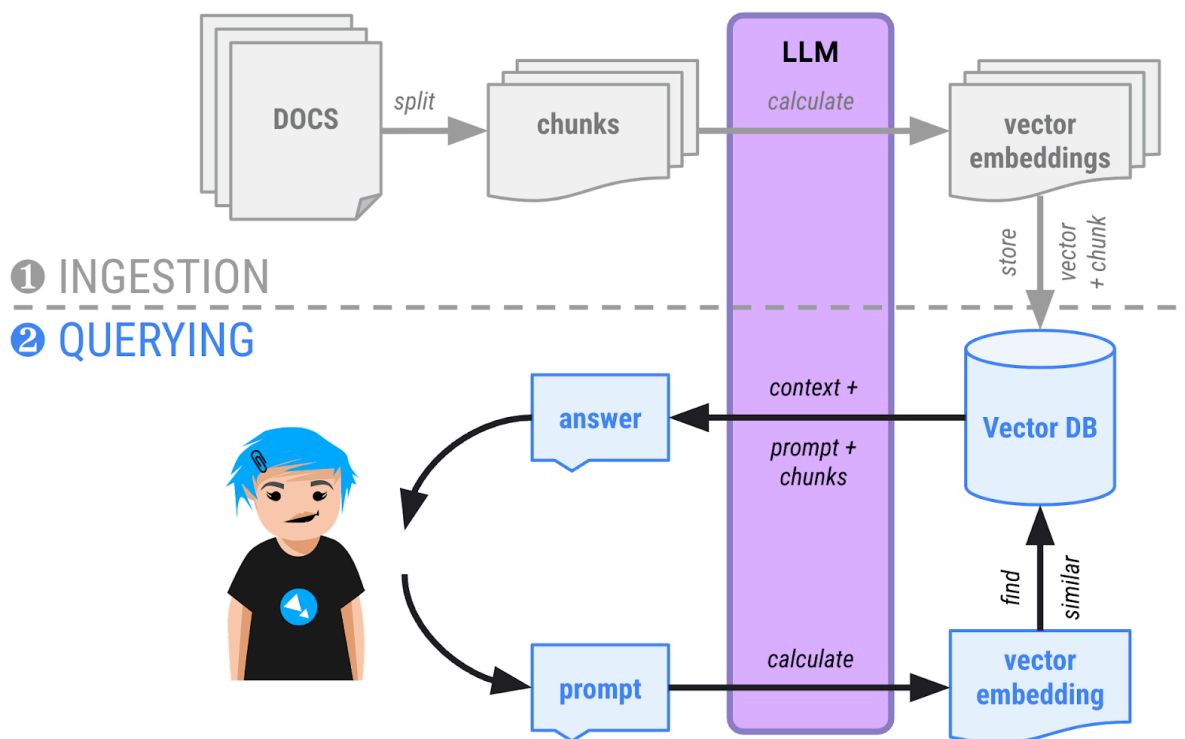
這些資料庫專門用於向量儲存和運算。大多數傳統資料庫 (例如 PostgreSQL 等) 通常會提供擴充功能，可擴充資料庫以允許向量儲存和計算。

在 RAG 中，有兩個階段：

1. **擷取階段**：將文件載入記憶體、分割成較小的分塊，並計算向量嵌入 (分塊的高多維度向量表示法)，然後儲存在可執行語意搜尋的向量資料庫中。通常需要在需要將新文件新增至文件集合時，才會執行一次這項擷取作業。



2. **查詢階段**：使用者現在可以詢問文件相關問題。問題也會轉換為向量，並與資料庫中的所有其他向量進行比較。最相似的向量通常在語意上相關，且由向量資料庫傳回。接著，系統會將對話脈絡、與資料庫傳回向量相應的文字區塊提供給 LLM，並要求 LLM 根據這些區塊提供答案。



準備文件

在這個新範例中，您將詢問有關虛構車輛製造商所生產的虛構車款：Cymbal Starlight！也就是說，模型不應包含虛構車輛的文件。因此，如果 Gemini 能正確回答有關這輛車的問題，就表示 RAG 方法有效，可以搜尋您的文件。

導入聊天機器人

讓我們瞭解如何建構兩階段方法：首先是文件擷取，然後是查詢時間 (也稱為「擷取階段」)，也就是使用者詢問文件相關問題時。

在本範例中，這兩個階段都是在同一個類別中實作。一般來說，您會使用一個應用程式處理擷取作業，另一個應用程式則為使用者提供聊天機器人介面。

此外，在本範例中，我們將使用記憶體內向量資料庫。在實際的生產環境中，擷取和查詢階段會分成兩個不同的應用程式，而向量會保留在獨立資料庫中。

注意：完整原始碼位於 `app/src/main/java/gemini/workshop` 目錄的 `RAG.java` 中

文件匯入

文件擷取階段的第一步，是找出有關虛構車輛的 PDF 檔案，並準備 PdfParser 讀取該檔案：

```
URL url = new URI("https://raw.githubusercontent.com/meteatamel/genai-beyond-basics/main/samples/grounding/vertexai-search/cymbal-starlight-2024.pdf").toURL();
ApachePdfBoxDocumentParser pdfParser = new ApachePdfBoxDocumentParser();
Document document = pdfParser.parse(url.openStream());
```

您要建立的是**嵌入模型**的執行個體，而不是先建立一般的聊天語言模型。這個模型專門負責建立文字片段 (字詞、句子或段落) 的向量表示法。這項功能會傳回浮點數向量，而非文字回覆。

```
VertexAiEmbeddingModel embeddingModel = VertexAiEmbeddingModel.builder()
    .endpoint(System.getenv("LOCATION") + "-aiplatform.googleapis.com:443")
    .project(System.getenv("PROJECT_ID"))
    .location(System.getenv("LOCATION"))
    .publisher("google")
    .modelName("text-embedding-005")
    .maxRetries(3)
    .build();
```

資訊：這個範例使用 `text-embedding-005` 模型計算向量嵌入。如要進一步瞭解嵌入和模型，請參閱[文字嵌入說明文件](#)。

警告：請注意，在擷取階段和查詢階段，請務必使用完全相同的嵌入模型。不同的嵌入模型，甚至是同一嵌入模型不同版本，都可能在產生的浮點向量中提供非常不同的值。在這種情況下，相似度計算結果就不會準確。這就像是在不同的地圖上使用不同的測量單位。

接著，您需要幾個類別來共同完成下列事項：

- 載入 PDF 文件並分割成多個區塊。
- 為所有這些區塊建立向量嵌入。

```
InMemoryEmbeddingStore<TextSegment> embeddingStore =
    new InMemoryEmbeddingStore<>();

EmbeddingStoreIngestor storeIngestor = EmbeddingStoreIngestor.builder()
    .documentSplitter(DocumentSplitters.recursive(500, 100))
    .embeddingModel(embeddingModel)
    .embeddingStore(embeddingStore)
    .build();
storeIngestor.ingest(document);
```

系統會建立 `InMemoryEmbeddingStore` 的例項 (記憶體內向量資料庫)，用於儲存向量嵌入。

注意：在這個範例中，為求簡單，我們使用記憶體內向量資料庫，但在實際運作時，您會想使用 Google Cloud Platform 提供的資料庫，例如 [PostgreSQL 適用的 Cloud SQL](#)、[AlloyDB](#) 或 [BigQuery](#)。所有格式都支援向量。

由於 `DocumentSplitters` 類別，文件會分割成多個區塊。系統會將 PDF 檔案的文字分割成 500 個字元的片段，並重疊 100 個字元 (與下一個區塊，避免將字詞或句子切成片段)。

商店擷取器會連結文件分割器、用於計算向量的嵌入模型，以及記憶體內向量資料庫。接著，`ingest()` 方法會負責執行攝入作業。

第一階段已結束，文件已轉換為文字區塊，並與相關聯的向量嵌入項目一起儲存在向量資料庫中。

提問

現在可以開始提問了！建立即時通訊模型來展開對話：

```
ChatLanguageModel model = VertexAiGeminiChatModel.builder()
    .project(System.getenv("PROJECT_ID"))
    .location(System.getenv("LOCATION"))
    .modelName("gemini-2.0-flash")
    .maxOutputTokens(1000)
    .build();
```

您還需要檢索器類別，將向量資料庫 (位於 `embeddingStore` 變數中) 連結至嵌入模型。這項工具的工作是計算使用者查詢的向量嵌入，藉此查詢向量資料庫，找出資料庫中相似的向量：

```
EmbeddingStoreContentRetriever retriever =
    new EmbeddingStoreContentRetriever(embeddingStore, embeddingModel);
```

建立代表車輛專家助理的介面，也就是 `AiServices` 類別將實作的介面，方便您與模型互動：

```
interface CarExpert {
    Result<String> ask(String question);
}
```

`CarExpert` 介面會傳回以 `LangChain4j` 的 `Result` 類別包裝的字串回應。為什麼要使用這個包裝函式？因為這項工具不僅會提供答案，還會讓您檢查內容擷取工具從資料庫傳回的區塊。這樣一來，您就能向使用者顯示用來生成最終答案的文件來源。

此時，您可以設定新的 AI 服務：

```
CarExpert expert = AiServices.builder(CarExpert.class)
    .chatLanguageModel(model)
    .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
    .contentRetriever(retriever)
    .build();
```

這項服務會將下列項目繫結在一起：

- 您先前設定的聊天語言模型。
- 對話記憶，可追蹤對話內容。
- 檢索器會比較向量嵌入查詢與資料庫中的向量。

注意：系統會使用[預設提示範本](#)，指示聊天模型根據向量資料庫中提供的相關文件摘錄內容生成回覆。但您可以覆寫及自訂這個提示範本，方法是使用下列設定擴充上述 `AiServices`：

```

.retrievalAugmentor(DefaultRetrievalAugmentor.builder()
    .contentInjector(DefaultContentInjector.builder()
        .promptTemplate(PromptTemplate.from("""
            You are an expert in car automotive, and you answer concisely.

            Here is the question: {{userMessage}}

            Answer using the following information:
            {{contents}}
            """))
    .build())
.contentRetriever(retriever)
.build()

```

現在可以開始提問了！

```

List.of(
    "What is the cargo capacity of Cymbal Starlight?",
    "What's the emergency roadside assistance phone number?",
    "Are there some special kits available on that car?"
).forEach(query -> {
    Result<String> response = expert.ask(query);
    System.out.printf("%n=== %s === %n%n %s %n%n", query, response.content());
    System.out.println("SOURCE: " + response.sources().getFirst().textSegment().text());
});

```

完整原始碼位於 `app/src/main/java/gemini/workshop` 目錄的 `RAG.java` 中。

執行範例：

```
./gradlew -q run -DjavaMainClass=gemini.workshop.RAG
```

注意：如果收到 `RESOURCE_EXHAUSTED` 錯誤，表示您已超出 VertexAI API 的配額。您可以點選錯誤訊息中的連結來提高配額，或將 `modelName` 變更為 `gemini-2.0-flash-lite` 等。

輸出內容應會顯示問題的答案：

=== What is the cargo capacity of Cymbal Starlight? ===

The Cymbal Starlight 2024 has a cargo capacity of 13.5 cubic feet.

SOURCE: Cargo

The Cymbal Starlight 2024 has a cargo capacity of 13.5 cubic feet. The cargo area is located in the trunk of the vehicle.

To access the cargo area, open the trunk lid using the trunk release lever located in the driver's footwell.

When loading cargo into the trunk, be sure to distribute the weight evenly. Do not overload the trunk, as this could affect the vehicle's handling and stability.

Luggage

=== What's the emergency roadside assistance phone number? ===

The emergency roadside assistance phone number is 1-800-555-1212.

SOURCE: Chapter 18: Emergencies

Roadside Assistance

If you experience a roadside emergency, such as a flat tire or a dead battery, you can call roadside

assistance for help. Roadside assistance is available 24 hours a day, 7 days a week.

To call roadside assistance, dial the following number:

1-800-555-1212

When you call roadside assistance, be prepared to provide the following information:

Your name and contact information

Your vehicle's make, model, and year

Your vehicle's location

=== Are there some special kits available on that car? ===

Yes, the Cymbal Starlight comes with a tire repair kit.

SOURCE: Lane keeping assist: This feature helps to keep you in your lane by gently steering the vehicle back into the lane if you start to drift.

Adaptive cruise control: This feature automatically adjusts your speed to maintain a safe following

distance from the vehicle in front of you.

Forward collision warning: This feature warns you if you are approaching another vehicle too quickly.

Automatic emergency braking: This feature can automatically apply the brakes to avoid a collision.

注意：系統只會顯示第一個來源區塊，因此使用者問題的答案可能不在這個區塊中，但可能位於未顯示的其他區塊。您可以視需要更新範例，顯示向量資料庫傳回的其他區塊。

11. 函式呼叫

在某些情況下，您會希望 LLM 存取外部系統，例如擷取資訊或執行動作的遠端網路 API，或是執行某種運算的服務。例如：

遠端網頁 API：

- 追蹤及更新顧客訂單。
- 在問題追蹤工具中找出或建立票證。
- 擷取即時資料，例如股票報價或 IoT 感應器測量結果。
- 傳送電子郵件。

計算工具：

- 計算機，可處理更進階的數學問題。
- 程式碼解讀：在大型語言模型需要推理邏輯時執行程式碼。
- 將自然語言要求轉換為 SQL 查詢，讓 LLM 可以查詢資料庫。

函式呼叫 (有時稱為工具或工具使用) 是指模型可以代表使用者要求呼叫一或多個函式，以便使用最新資料正確回答使用者提示。

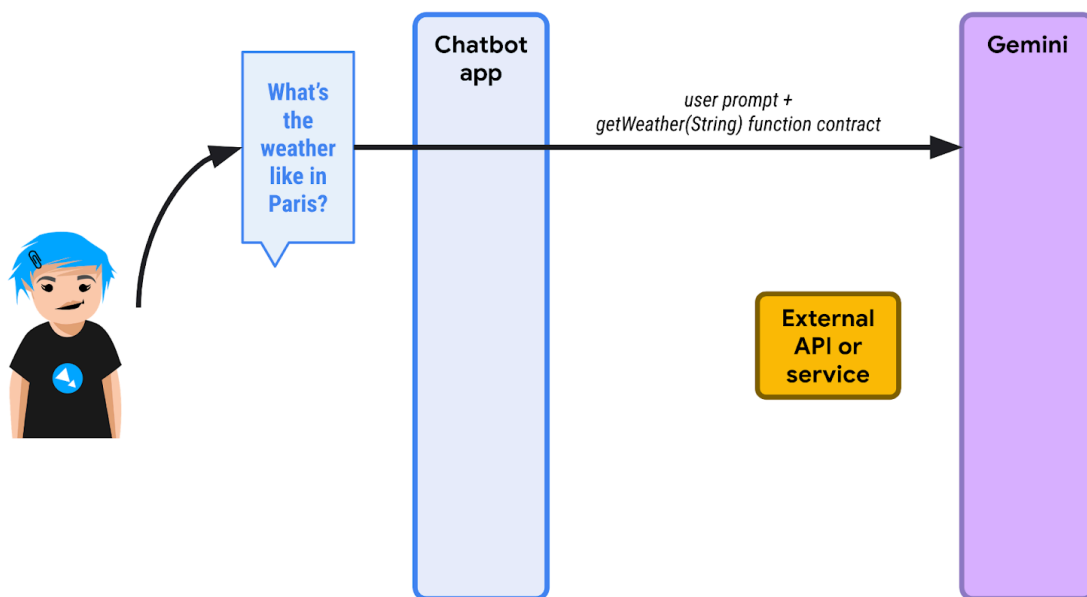
如果使用者提供特定提示，且大型語言模型知道現有函式可能與該情境相關，就能回覆函式呼叫要求。整合 LLM 的應用程式接著可以代表使用者呼叫函式，然後回覆 LLM，而 LLM 則會解讀回覆內容，並以文字形式提供答案。

函式呼叫的四個步驟

我們來看看函式呼叫的範例：取得天氣預報資訊。

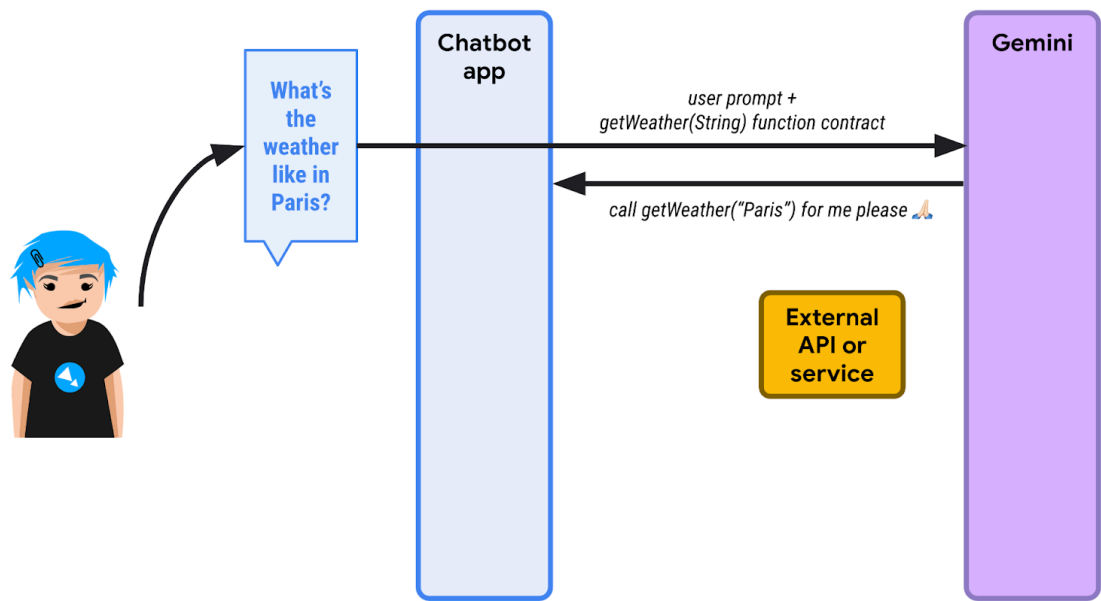
如果你向 Gemini 或任何其他 LLM 詢問巴黎的天氣，系統會回覆目前沒有天氣預報資訊。如要讓 LLM 即時存取天氣資料，您需要定義一些可要求使用的函式。

請參閱下圖：



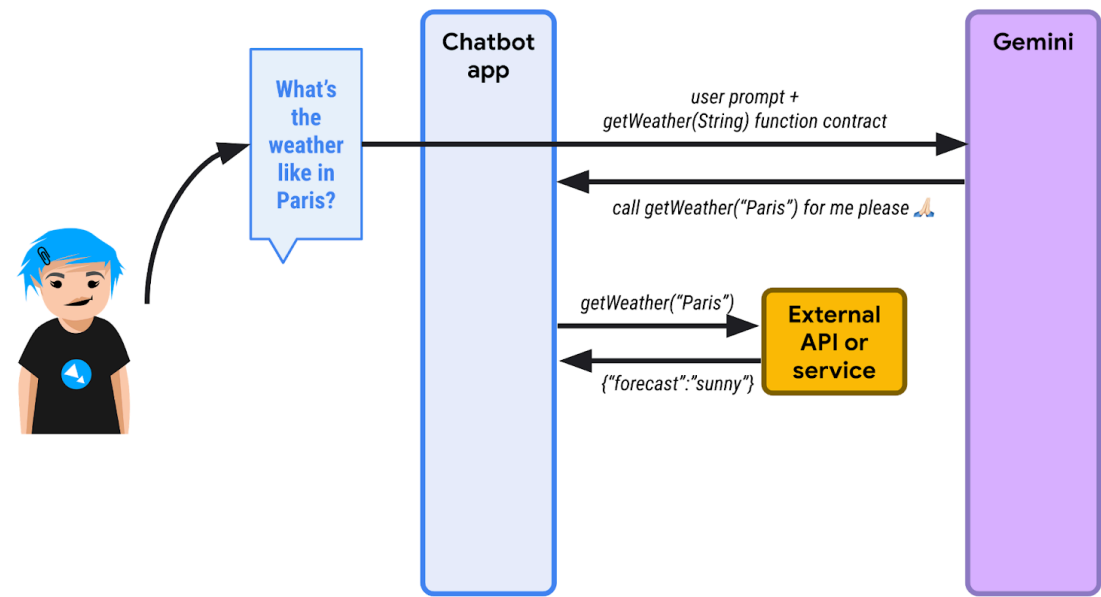
1 首先，使用者詢問巴黎的天氣。聊天機器人應用程式 (使用 LangChain4j) 知道有一或多個函式可供使用，協助 LLM 滿足查詢需求。聊天機器人會傳送初始提示，以及可呼叫的函式清單。這裡的函式名為 `getWeather()`，會採用位置的字

串參數。

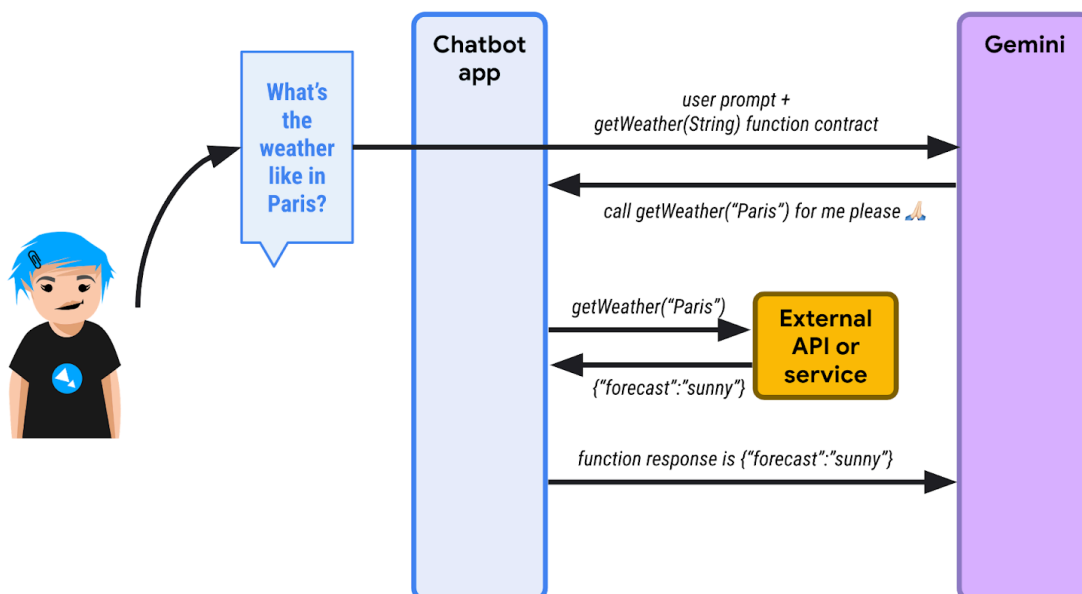


由於 LLM 不瞭解天氣預報，因此不會以文字回覆，而是傳回函式執行要求。聊天機器人必須呼叫 `getWeather()` 函式，並將 "Paris" 做為位置參數。

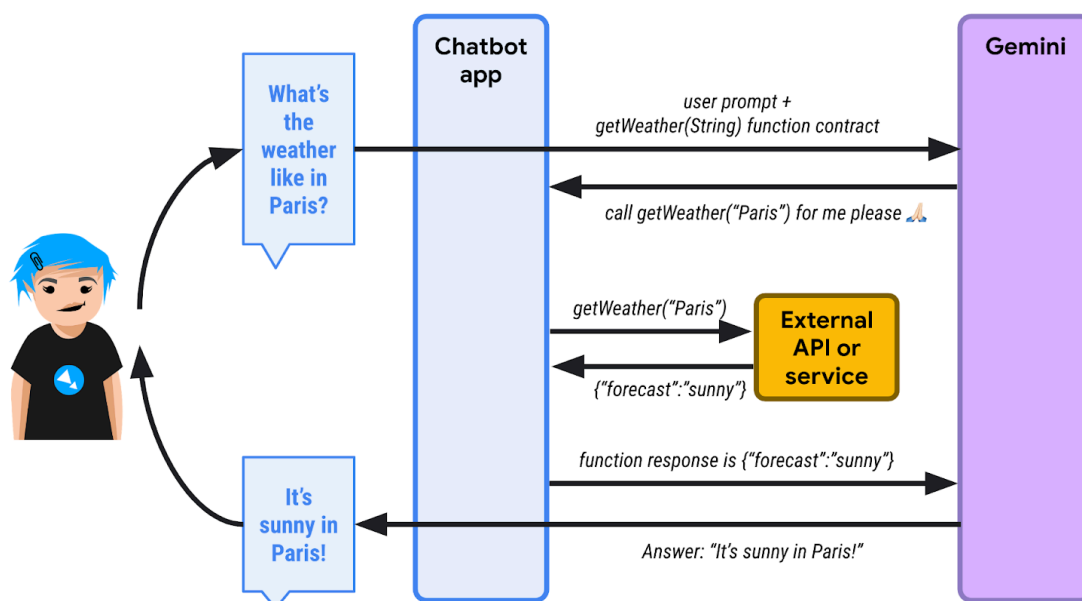
2 聊天機器人會代表 LLM 叫用該函式，並擷取函式回應。假設回覆內容為 `{"forecast": "sunny"}`。



3 聊天機器人應用程式將 JSON 回應傳回 LLM。



4 LLM 會查看 JSON 回應、解讀該資訊，並最終回覆巴黎天氣晴朗。



注意：「函式呼叫」一詞可能會造成誤解，因為這表示模型本身會執行函式。但事實並非如此。模型只會要求呼叫函式，整合 LLM 的應用程式有責任進行該呼叫，並將其傳回模型。

以程式碼形式呈現每個步驟

注意：完整原始碼位於 `app/src/main/java/gemini/workshop` 目錄的 [FunctionCalling.java](#) 中

首先，請照常設定 Gemini 模型：

```
ChatLanguageModel model = VertexAiGeminiChatModel.builder()
    .project(System.getenv("PROJECT_ID"))
    .location(System.getenv("LOCATION"))
    .modelName("gemini-2.0-flash")
    .maxOutputTokens(100)
    .build();
```

定義工具規格，說明可呼叫的函式：

```
ToolSpecification weatherToolSpec = ToolSpecification.builder()
    .name("getWeather")
    .description("Get the weather forecast for a given location or city")
    .parameters(JsonObjectSchema.builder()
        .addStringProperty(
            "location",
            "the location or city to get the weather forecast for")
        .build())
    .build();
```

系統會定義函式名稱，以及參數的名稱和類型，但請注意，函式和參數都會提供說明。說明非常重要，可協助 LLM 真正瞭解函式的功能，進而判斷是否需要在對話情境中呼叫該函式。

首先，請傳送關於巴黎天氣的初始問題：

```
List<ChatMessage> allMessages = new ArrayList<>();

// 1) Ask the question about the weather
UserMessage weatherQuestion = UserMessage.from("What is the weather in Paris?");
allMessages.add(weatherQuestion);
```

在步驟 2 中，我們傳遞希望模型使用的工具，模型會回覆工具執行要求：

```
// 2) The model replies with a function call request
Response<AiMessage> messageResponse = model.generate(allMessages, weatherToolSpec);
ToolExecutionRequest toolExecutionRequest =
    messageResponse.content().toolExecutionRequests().getFirst();
System.out.println("Tool execution request: " + toolExecutionRequest);
allMessages.add(messageResponse.content());
```

步驟 3：此時，我們知道大型語言模型希望我們呼叫哪個函式。在程式碼中，我們不會實際呼叫外部 API，只會直接傳回假設的天氣預報：

```
// 3) We send back the result of the function call
ToolExecutionResultMessage toolExecResMsg =
    ToolExecutionResultMessage.from(toolExecutionRequest,
        "{\n\"location\": \"Paris\", \"forecast\": \"sunny\", \"temperature\": 20}");
allMessages.add(toolExecResMsg);
```

在步驟 4 中，LLM 會瞭解函式執行結果，然後合成文字回覆：


```
// 4) The model answers with a sentence describing the weather
Response<AiMessage> weatherResponse = model.generate(allMessages);
System.out.println("Answer: " + weatherResponse.content().text());
```

完整原始碼位於 `app/src/main/java/gemini/workshop` 目錄的 `FunctionCalling.java` 中。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.FunctionCalling
```

畫面會顯示類似以下的輸出：

```
Tool execution request: ToolExecutionRequest { id = null, name = "getWeatherForecast",
arguments = "{\"location\":\"Paris\"}" }
Answer: The weather in Paris is sunny with a temperature of 20 degrees Celsius.
```

如上方的輸出內容所示，您可以看到工具執行要求和答案。

12. LangChain4j 會處理函式呼叫

在上一個步驟中，您已瞭解一般文字問題/答案和函式要求/回應互動如何交錯，以及如何直接提供要求的函式回應，而不必呼叫實際函式。

不過，LangChain4j 也提供更高層次的抽象機制，可為您透明地處理函式呼叫，同時照常處理對話。

單一函式呼叫

我們來逐一看看 `FunctionCallingAssistant.java`。

首先，請建立代表函式回應資料結構的記錄：

```
record WeatherForecast(String location, String forecast, int temperature) {}
```

回覆內容包含地點、天氣預報和溫度資訊。

接著，建立包含您要提供給模型的實際函式的類別：

```
static class WeatherForecastService {
    @Tool("Get the weather forecast for a location")
    WeatherForecast getForecast(@P("Location to get the forecast for") String location) {
        if (location.equals("Paris")) {
            return new WeatherForecast("Paris", "Sunny", 20);
        } else if (location.equals("London")) {
            return new WeatherForecast("London", "Rainy", 15);
        } else {
            return new WeatherForecast("Unknown", "Unknown", 0);
        }
    }
}
```

請注意，這個類別只包含單一函式，但會以 `@Tool` 註解加上註解，對應模型可要求呼叫的函式說明。

函式的參數 (這裡只有一個) 也會加上註解，但會使用簡短的 `@P` 註解，其中也會提供參數說明。您可以視需要新增任意數量的函式，供模型使用，以因應更複雜的情況。

在這個類別中，您會傳回一些罐頭回應，但如果您想呼叫真正的外部天氣預報服務，則會在該方法的主體中呼叫該服務。

如您在先前的做法中建立 `ToolSpecification` 時所見，請務必記錄函式的功能，並說明參數對應的內容。這有助於模型瞭解如何及何時使用這項函式。

注意：採用這種替代方法時，LangChain4j 架構會在收到模型傳送的函式呼叫要求後，負責啟動函式呼叫。您不必手動呼叫函式或提供函式執行結果。LangChain4j 會自動處理這項作業。

接著，LangChain4j 可讓您提供與要用來與模型互動的合約相應的介面。這個介面很簡單，會接收代表使用者訊息的字串，並傳回對應模型回覆的字串：

```
interface WeatherAssistant {
    String chat(String userMessage);
}
```

如要處理更進階的情況，也可以使用更複雜的簽章，當中包含 `LangChain4j` 的 `UserMessage` (適用於使用者訊息) 或 `AiMessage` (適用於模型回應)，甚至是 `TokenStream`，因為這些較複雜的物件也包含額外資訊，例如消耗的權杖數量等。但為了簡化，我們只會將字串做為輸入內容，並將字串做為輸出內容。

最後，請使用 `main()` 方法將所有部分整合在一起：

```
public static void main(String[] args) {
    ChatLanguageModel model = VertexAiGeminiChatModel.builder()
        .project(System.getenv("PROJECT_ID"))
        .location(System.getenv("LOCATION"))
        .modelName("gemini-2.0-flash")
        .build();

    WeatherForecastService weatherForecastService = new WeatherForecastService();

    WeatherAssistant assistant = AiServices.builder(WeatherAssistant.class)
        .chatLanguageModel(model)
        .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
        .tools(weatherForecastService)
        .build();

    System.out.println(assistant.chat("What is the weather in Paris?"));
    System.out.println(assistant.chat("Is it warmer in London or in Paris?"));
}
```

如常設定 Gemini 聊天模型。接著，您會將天氣預報服務例項化，其中包含模型要求我們呼叫的「函式」。

現在，請再次使用 `AiServices` 類別，繫結對話模型、對話記憶體和工具 (即天氣預報服務及其函式)。 `AiServices` 會傳回實作您定義的 `WeatherAssistant` 介面的物件。最後只要呼叫該助理的 `chat()` 方法即可。呼叫時，您只會看到文字回覆，但開發人員不會看到函式呼叫要求和函式呼叫回覆，系統會自動處理這些要求，且過程完全透明。如果 Gemini 認為應呼叫函式，就會回覆函式呼叫要求，而 `LangChain4j` 會代表您呼叫本機函式。

執行範例：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.FunctionCallingAssistant
```

畫面會顯示類似以下的輸出：

```
OK. The weather in Paris is sunny with a temperature of 20 degrees.

It is warmer in Paris (20 degrees) than in London (15 degrees).
```

以上是單一函式的範例。

多次呼叫函式

您也可以使用多個函式，並讓 LangChain4j 代表您處理多個函式呼叫。如需多個函式的範例，請參閱 [MultiFunctionCallingAssistant.java](#)。

可轉換貨幣：

```
@Tool("Convert amounts between two currencies")
double convertCurrency(
    @P("Currency to convert from") String fromCurrency,
    @P("Currency to convert to") String toCurrency,
    @P("Amount to convert") double amount) {

    double result = amount;

    if (fromCurrency.equals("USD") && toCurrency.equals("EUR")) {
        result = amount * 0.93;
    } else if (fromCurrency.equals("USD") && toCurrency.equals("GBP")) {
        result = amount * 0.79;
    }

    System.out.println(
        "convertCurrency(fromCurrency = " + fromCurrency +
            ", toCurrency = " + toCurrency +
            ", amount = " + amount + ") == " + result);

    return result;
}
```

另一個取得股票價值的函式：

```
@Tool("Get the current value of a stock in US dollars")
double getStockPrice(@P("Stock symbol") String symbol) {
    double result = 170.0 + 10 * new Random().nextDouble();

    System.out.println("getStockPrice(symbol = " + symbol + ") == " + result);

    return result;
}
```

另一個函式，可將百分比套用至指定金額：

```

@Tool("Apply a percentage to a given amount")
double applyPercentage(@P("Initial amount") double amount, @P("Percentage between 0-100 to apply") double percentage) {
    double result = amount * (percentage / 100);

    System.out.println("applyPercentage(amount = " + amount + ", percentage = " + percentage + ") == " + result);

    return result;
}

```

接著，您可以結合所有這些函式和 MultiTools 類別，並提出「將 AAPL 股價的 10% 從美元換算為歐元是多少？」等問題。

```

public static void main(String[] args) {
    ChatLanguageModel model = VertexAiGeminiChatModel.builder()
        .project(System.getenv("PROJECT_ID"))
        .location(System.getenv("LOCATION"))
        .modelName("gemini-2.0-flash")
        .maxOutputTokens(100)
        .build();

    MultiTools multiTools = new MultiTools();

    MultiToolsAssistant assistant = AiServices.builder(MultiToolsAssistant.class)
        .chatLanguageModel(model)
        .chatMemory(withMaxMessages(10))
        .tools(multiTools)
        .build();

    System.out.println(assistant.chat(
        "What is 10% of the AAPL stock price converted from USD to EUR?"));
}

```

執行方式如下：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.MultiFunctionCallingAssistant
```

您應該會看到呼叫的多個函式：

```

getStockPrice(symbol = AAPL) == 172.8022224055534
convertCurrency(fromCurrency = USD, toCurrency = EUR, amount = 172.8022224055534) == 160.70606683716468
applyPercentage(amount = 160.70606683716468, percentage = 10.0) == 16.07060668371647
10% of the AAPL stock price converted from USD to EUR is 16.07060668371647 EUR.

```

服務專員

函式呼叫是 Gemini 等大型語言模型的絕佳擴充機制。這讓我們得以建構更複雜的系統，通常稱為「代理程式」或「AI 助理」。這些代理可以透過外部 API 與外部世界互動，也可以與可能對外部環境產生副作用的服務互動 (例如傳送電子郵件、建立工單等)。

建立這類功能強大的代理程式時，請務必負責任地進行。建議您在採取自動動作前，先考慮加入人工審查程序。設計與外部世界互動的 LLM 輔助代理程式時，請務必將安全措施納入考量。

13. 使用 Ollama 和 TestContainers 執行 Gemma

到目前為止，我們一直使用 Gemini，但還有一個小妹模型 [Gemma](#)。

[Gemma](#) 是一系列先進的輕量級開放式模型，採用與建立 Gemini 模型時相同的研究成果和技術。最新的 Gemma 模型是 Gemma3，提供四種大小：1B (*僅限文字*)、4B、12B 和 27B。這些模型可免費取得權重，而且體積小巧，因此您可以在自己的裝置上執行，即使是筆電或 Cloud Shell 也不例外。

如何執行 Gemma？

執行 Gemma 的方式有很多種：在雲端執行、透過 Vertex AI 點按按鈕執行，或使用某些 GPU 在 GKE 執行，但您也可以在本機執行。

如要在本機執行 Gemma，建議使用 [Ollama](#)。這項工具可讓您在本機執行小型模型，例如 Llama、Mistral 和許多其他模型。這與 Docker 類似，但適用於 LLM。

按照作業系統的[說明](#)安裝 Ollama。

如果您使用 Linux 環境，安裝 Ollama 後必須先啟用。

```
ollama serve > /dev/null 2>&1 &
```

在本機安裝後，您就可以執行指令來提取模型：

```
ollama pull gemma3:1b
```

等待模型提取完成。這項作業可能需要一點時間。

執行模型：

```
ollama run gemma3:1b
```

現在你可以與模型互動：

```
>>> Hello!
Hello! It's nice to hear from you. What can I do for you today?
```

如要退出提示，請按下 Ctrl+D 鍵

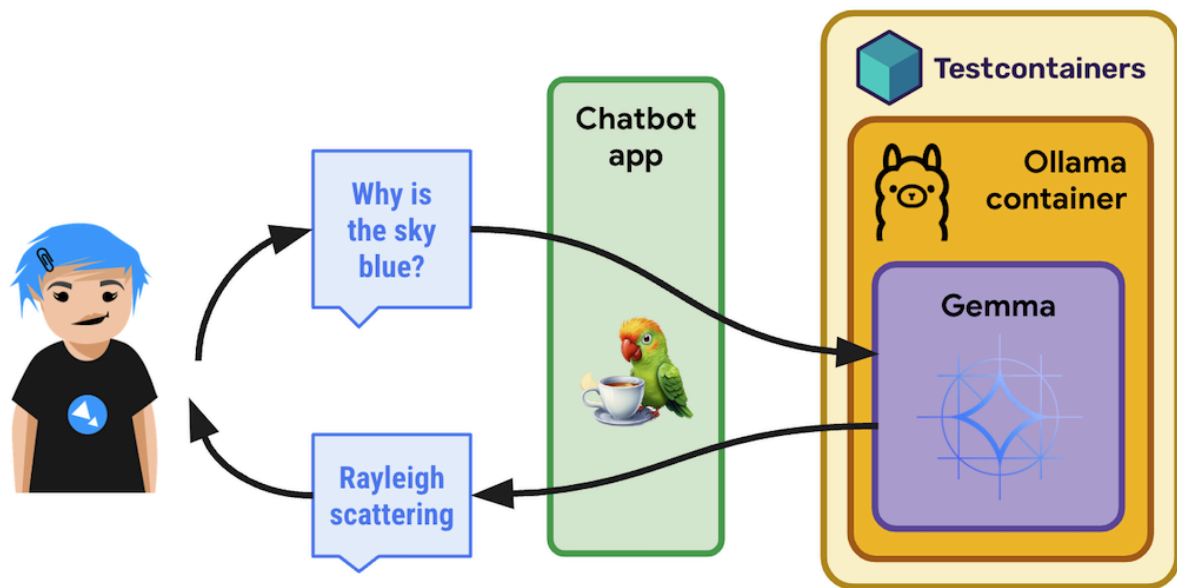
注意：在本範例中，我們使用的是 10 億參數的 Gemma3 (*僅限文字*)。這是小型模型。如要試用較大的模型 (例如 Gemma3 27B)，請確保工作站有足夠的磁碟空間。在 [Ollama 說明文件](#) 中查看可用模型及其檔案大小

在 TestContainers 上以 Ollama 執行 Gemma

您可以使用 [TestContainers](#) 處理的容器，在容器內使用 Ollama，不必在本機安裝及執行 Ollama。

[TestContainers](#) 不僅適用於測試，還能用於執行容器。你甚至可以善用特定 [OllamaContainer](#)！

以下是整體情況：



導入

我們來逐一看看 `GemmaWithOllamaContainer.java`。

首先，您需要建立衍生 Ollama 容器，並提取 Gemma 模型。這個映像檔可能已存在 (先前執行作業時建立)，也可能即將建立。如果映像檔已存在，您只要告知 TestContainers 要以 Gemma 支援的變體取代預設的 Ollama 映像檔即可：


```

private static final String TC_OLLAMA_GEMMA3 = "tc-ollama-gemma3-1b";
public static final String GEMMA_3 = "gemma3:1b";

// Creating an Ollama container with Gemma 3 if it doesn't exist.
private static OllamaContainer createGemmaOllamaContainer() throws IOException,
InterruptedException {

    // Check if the custom Gemma Ollama image exists already
    List<Image> listImagesCmd = DockerClientFactory.lazyClient()
        .listImagesCmd()
        .withImageNameFilter(TC_OLLAMA_GEMMA3)
        .exec();

    if (listImagesCmd.isEmpty()) {
        System.out.println("Creating a new Ollama container with Gemma 3 image...");
        OllamaContainer ollama = new OllamaContainer("ollama/ollama:0.7.1");
        System.out.println("Starting Ollama...");
        ollama.start();
        System.out.println("Pulling model...");
        ollama.execInContainer("ollama", "pull", GEMMA_3);
        System.out.println("Committing to image...");
        ollama.commitToImage(TC_OLLAMA_GEMMA3);
        return ollama;
    }

    System.out.println("Ollama image substitution...");
    // Substitute the default Ollama image with our Gemma variant
    return new OllamaContainer(
        DockerImageName.parse(TC_OLLAMA_GEMMA3)
            .asCompatibleSubstituteFor("ollama/ollama"));
}

```

接著，您要建立並啟動 Ollama 測試容器，然後指向要使用的模型所在容器的位址和連接埠，建立 Ollama 即時通訊模型。最後，只要照常叫用 `model.generate(yourPrompt)` 即可：

```

public static void main(String[] args) throws IOException, InterruptedException {
    OllamaContainer ollama = createGemmaOllamaContainer();
    ollama.start();

    ChatLanguageModel model = OllamaChatModel.builder()
        .baseUrl(String.format("http://%s:%d", ollama.getHost(),
ollama.getFirstMappedPort()))
        .modelName(GEMMA_3)
        .build();

    String response = model.generate("Why is the sky blue?");

    System.out.println(response);
}

```

執行方式如下：

```
./gradlew run -q -DjavaMainClass=gemini.workshop.GemmaWithOllamaContainer
```

第一次執行時，系統需要一段時間建立及執行容器，完成後您應該會看到 Gemma 回覆：

```
INFO: Container ollama/ollama:0.7.1 started in PT7.228339916S
The sky appears blue due to Rayleigh scattering. Rayleigh scattering is a phenomenon that occurs when sunlight interacts with molecules in the Earth's atmosphere.

* **Scattering particles:** The main scattering particles in the atmosphere are molecules of nitrogen (N2) and oxygen (O2).
* **Wavelength of light:** Blue light has a shorter wavelength than other colors of light, such as red and yellow.
* **Scattering process:** When blue light interacts with these molecules, it is scattered in all directions.
* **Human eyes:** Our eyes are more sensitive to blue light than other colors, so we perceive the sky as blue.

This scattering process results in a blue appearance for the sky, even though the sun is actually emitting light of all colors.

In addition to Rayleigh scattering, other atmospheric factors can also influence the color of the sky, such as dust particles, aerosols, and clouds.
```

您已在 Cloud Shell 中執行 Gemma !

14. 恭喜

恭喜！您已使用 LangChain4j 和 Gemini API，以 Java 成功建構第一個生成式 AI 即時通訊應用程式。您在過程中發現，多模態大型語言模型功能強大，能夠處理各種工作，例如問答，甚至可處理您自己的說明文件、資料擷取、與外部 API 互動等。

後續步驟

現在輪到您透過強大的 LLM 整合功能，提升應用程式效能！

其他資訊

- [生成式 AI 的常見用途](#)
- [生成式 AI 訓練資源](#)
- [透過 Generative AI Studio 與 Gemini 互動](#)
- [負責任的 AI 技術](#)

參考文件

- [Vertex AI 生成式 AI 說明文件](#)
 - [GitHub 上的 LangChain4j](#)
-